

Llumnix: Dynamic Scheduling for Large Language Model Serving

1 Paper summary

This paper presents Llumnix, which is a scheduling system for LLM serving that focuses on managing requests across multiple model instances, not just optimizing one instance at a time. The main issue is that LLM workloads are very unpredictable: requests can have very different prompt lengths, output lengths, priorities, and memory usage. Because of this, simple one-time scheduling decisions often don't work very well.

Most previous systems mainly focus on improving throughput inside a single instance, but they don't handle cluster-level problems like load imbalance, memory fragmentation, or interference between requests very well. Llumnix tries to solve this by allowing requests to be moved between instances while they are already running, kind of similar to process migration in operating systems.

To make this possible, the paper introduces a live migration mechanism that can move both the request and its GPU memory state with very little downtime. Then on top of that, they build a distributed scheduler that dynamically decides where requests should run based on something they call virtual usage.

This helps the system handle different goals at once, like balancing load, reducing fragmentation, supporting priorities, and reacting better to autoscaling events.

The evaluation on a 16-GPU cluster shows pretty strong improvements compared to other systems, especially for tail latency. They report up to 15x better P99 first-token latency and lower serving cost in some cases. Overall, the main idea is that LLM serving should be treated more like a dynamic scheduling problem instead of only an inference optimization problem.

2 Contributions

The first contribution is identifying LLM serving as a dynamic multi-instance scheduling problem. This is important because the paper shifts the focus away from only kernel-level optimization and looks more at cluster-level behavior like isolation, fragmentation, and priorities.

The second contribution is runtime rescheduling with live migration. Requests can move between model instances even after execution has started, which allows the system to adapt to changing workloads instead of relying only on the initial placement decision.

The third contribution is the distributed scheduling framework and virtual-usage policy. The same mechanism is used for several goals at once, like load balancing, priority handling, and reducing fragmentation. This makes the system more flexible overall, even if it also increases complexity.

3 Problems

One limitation is that Llumnix is quite complex compared to simpler serving systems. Supporting migration, distributed scheduling, and GPU-memory coordination adds a lot of moving parts, which could make deployment and debugging harder in practice.

Another issue is that the benefits probably depend on workload scale and variability. In smaller deployments, or in cases where requests are more predictable, the extra complexity and migration overhead might not be worth it.

A third point is that the system tries to optimize several things at the same time, like latency, cost, and prioritization, but these goals can sometimes conflict. The virtual-

usage policy looks elegant, but in practice it may still require careful tuning depending on the workload and what the operator cares about most.